

OOP

目录

OOP.....	1
○ 哇好神秘还有绪论课 2026-02-28	1
壹 编译源程序 2026-03-03	1
一、编译链接	1
二、Make	2
三、函数重载：一个函数名字，两个以上实现方法。	3
四、auto, decltype	3
auto	3
decltype	3
五、内存申请和释放	4
贰 对象 2026-03-10	4
一、访问权限：	4
二、this 指针	5
三、内联函数	5
叁 构造函数和析构函数 2026-03-17	5
一、构造函数：诞生	5
委派构造函数：调用其他构造函数。	6
就地初始化：	6
默认/缺省构造函数：	6
对象数组的初始化：	6
二、析构函数：死亡	7
三、局部对象的构造和析构	7
四、全局对象的构造和析构	7

○ 哇好神秘还有绪论课 2026-02-28

我去，居然是雷霆方法论基础?! 结构化?! 面向对象?! 神秘神秘?!

不能反直觉反常识?! 强强!? 对象? 我的对象在哪里?! 弱弱,,, 属性和操作?! 静态和动态?! 抽象?!
过程抽象和数据抽象?! eg.类型是什么?! 数据的存储和表示+数据支持的操作?!

壹 编译源程序 2026-03-03

一、编译链接

举例： argv argc 等参数的运用

实际意义上的一个完整编译流程如下：

```
g++ -c ex1.cpp
g++ -o ex1 ex1.o
```

第一步是编译，第二步是链接。注意到还有多文件链接。

```
g++ -c ex1.cpp ex1.o
g++ -c func1.cpp func1.o
g++ ex1.o func1.o -o ex1
```

当然我们平时偷懒是可以的。只是想告诉你这个编译和链接的实际过程，第二步把两个文件里的函数链接了?!

链接文件里面声明和定义不相同会类型报错。#include 的本质是把代码直接复制下来了。声明可以多声明，定义只能定义一次。声明的时候可以修改只说参数类型，不喊名字。变量也可以声明或者定义。即使不赋值也可以被定义！注意下面的例子：

```
// num.cpp
int a=1;
```

```
// ex1.cpp
int a;
int main(){
    a=1;
    return 0
}
```

这时就炸了。重复定义。定义=声明+内存分配 ### 二、extern 关键字 有一点点烧脑。用于全局变量的共享。

```
//num.h
extern int a;
extern int b;
extern int c;
extern int d[10];

//num.cpp
#include<num.h>
a=1;
b=2;
c=3;
```

头文件中只声明，不定义，目的是防止重复定义。### 三、宏定义/常量 请使用 const 以保证类型的正确性！对于宏：

```
#define name(value) command_string
#define sqr(x) ((x)*(x))

sqr(3+2) = 25
```

请注意这个时候括号十分重要。因为不带括号的时候只是把串原封不动地搬运过来。宏定义可以用来防止头文件被反复包含。

```
// Method 1
#ifndef _HEAD_
#define _HEAD
...(如声明/定义)
#endif
// Method 2
#pragma once
```

还可以用来神秘的 debug。

```
#define DEBUG
#ifdef DEBUG
debug();
#endif
```

二、Make

Makefile 编写规则：只编译并链接被修改的文件，其他不管。但是如果头文件被改变了，所有引用了这个头文件的都要重新编译。格式举例：

```
all: main test
main: main.cpp stu.cpp
    g++ -o main main.cpp stu.cpp
test: stu.cpp stu_test.cpp
    g++ -o test stu_test.cpp stu.cpp
clean:
    rm main test

//bash 举例
make all
make test
make main
make clean
```

Make 的更多用法还是自己去查吧。氧气和臭氧优化后不能 gdb。

三、函数重载：一个函数名字，两个以上实现方法。

关键在于参数类型的不同。但是如果出现了参数的缺省，就会产生二义性。

举例：

```
int fun(int a, int b=0);
int fun(int a);
```

会产生二义性。因为当调用 `fun(1)` 的时候，编译器无法判断是调用第一个函数还是第二个函数。

但是！

```
int fun(int a);
int fun(float a);
```

就不会产生二义性。因为当调用 `fun(1)` 的时候，编译器会选择第一个函数，因为参数类型是 `int`，而不是 `float`。

四、auto, decltype

`auto` 和 `decltype` 都是 C++11 引入的类型推断机制，可以编译器根据上下文自动推断变量的类型。

auto

```
auto x = 5; // x 的类型被推断为 int
auto y = 3.14; // y 的类型被推断为 double
auto z = "Hello"; // z 的类型被推断为 const char*
```

- `auto` 必须在定义时初始化。
- `auto` 不能用于函数参数声明。
- `auto` 不能用如 `sizeof`, `typeid` 等上下文中。
- 在定义模板函数时用于声明依赖模板参数的返回类型：

```
template<typename _Tx, typename _Ty>
void Multiply(_Tx x, _Ty y){
    auto v = x * y; // 返回类型由编译器根据表达式 x * y 推断
    std::cout << v << std::endl;
}
Multiply(3, 4.5); // 输出 13.5, v 的类型被推断为 double
```

decltype

- 重用匿名类型：

```
struct {
    int a;
    double b;
} myStruct;
decltype(myStruct) anotherStruct; // anotherStruct 的类型与 myStruct 相同
```

除此之外还可以用来追踪推导 auto 返回的类型，在 C++ 14 后不必要：

```
auto func(int x,int y) -> decltype(x+y) {
    return x + y; // 返回类型被推断为 int
}
```

五、内存申请和释放

```
int* p = new int(10); // 申请一个 int 类型的内存
int *arr = new int[5]; // 申请一个 int 类型的数组
delete p; // 释放单个 int 类型的内存
delete[] arr; // 释放数组内存
```

nullptr 是 C++11 引入的空指针常量，表示一个空指针。使用 nullptr 可以避免与整数 0 混淆，提高代码的可读性和安全性。

贰 对象 2026-03-10

成员函数必须在类的内部声明，但是可以在类的外部定义（实现）。

举例：

```
class MyClass {
public:
    void myFunction(); // 成员函数声明
};
void MyClass::myFunction() { // 成员函数定义
    // 函数体
}
```

一、访问权限：

- public: 公共成员，类的外部可以访问。
- private: 私有成员，只有类的内部可以访问。也是默认访问权限。
- protected: 受保护成员，只有类的内部和派生类可以访问。

可以通过 object.member 的方式访问对象的成员函数和成员变量。或者 pointer->member 的方式访问指针指向对象的成员函数和成员变量。在类外只能访问 public 成员。

神秘：在类的内部可以访问传入的同一个类的对象的 private 成员！因为它们属于同一个类。

举例：

```
class MyClass {
private:
    int privateVar;
public:
    void setPrivateVar(int value) {
        privateVar = value; // 访问自己的 private 成员
    }
    void accessOther(MyClass& other) {
        other.privateVar = 10; // 访问同类的其他对象的 private 成员
    }
};
```

二、this 指针

指向当前对象的指针。可以在成员函数内部使用 `this` 来访问当前对象的成员变量和成员函数。

举例：

```
class MyClass {
private:
    int value;
public:
    MyClass(int v) : value(v) {}
    void printValue() {
        cout << "Value: " << this->value << endl;
    }
};
```

错综复杂的类外类内关系.....阿弥诺斯，小白手套欸呦喂，我这不赖，已取餐。

三、内联函数

由于和宏代码是直接拷贝复制代码到指定位置，会产生莫名其妙的边际效应，你永远无法想象到各种优先级问题，即使加了括号也不一定能完全解决。

相比之下，**内联函数**可以执行类型检查，可调试。它的本质是生成和函数等价的表达式。并且**宏定义**无法调用私有成员函数，而内联函数可以。

注意事项：

- 避免大段代码的内联，因为会增加编译时间和可执行文件的大小。
- 避免对包含循环、递归或复杂逻辑的函数使用内联。
- 避免将声明和定义分开，因为内联函数必须在调用之前定义。
- 定义在类内部的成员函数默认是内联的，但这只是一个建议，编译器可能会选择不内联。
- 一般构造函数、析构造函数通常会被编译器自动内联。

`inline` 永远只是建议修饰，不是命令。编译器有权拒绝不合理的内联请求。

叁 构造函数和析构造函数 2026-03-17

回顾：要对用户定义类型进行严格检查；要隐藏属性；要自动化对象的初始化和清除，做好对象内存分配等等。

一、构造函数：诞生

- 构造函数没有返回值
- 构造函数名与类名相同
- 构造函数可以重载

举例：

```
class Student {
    int ID;
public:
    Student(int id) : ID(id) {}
    Student(int year, int order){
        ID = year * 10000 + order;
    }
};
```

注意！按照声明顺序而不是初始化顺序排列成员！如下面 ID1 不可预测。

```
class Student {
    int ID1;
    int ID2;
public:
```

```
Student(int id) : ID2(id), ID1(ID2) {}
};
```

委派构造函数：调用其他构造函数。

```
class Info{
public:
    Info(int a):a(a){}
    Info(int a,int b):Info(a){b=b;}
    Info(int a,int b,int c):Info(a,b){c=c;}
};
```

就地初始化：

```
class A{
private:
    int a{0};
    double b=2.0;
public:
    A(){}
};
```

默认/缺省构造函数：

```
class A{
public:
    A(){};
};

A a;
A b=A();
A c();
// 这个时候 c 是一个函数声明，而不是一个对象声明。它可以生成一个对象。

//显式默认构造函数
class A{
public:
    A()=default;
};

//显式删除默认构造函数
class A{
public:
    A()=default;
    A(int i){}
    A(char cls)=delete;
};

A a('c');
```

一个实例生成的时候，会先调用成员的构造函数（如果成员没有就默认构造），再构造自己的。

如果你已经定义了构造函数，编译器不会帮你默认构造了！！！！

对象数组的初始化：

```
A a[50];
// 只有一个参数
A a[3]={1,2,3};
```

```
//多个参数
A a[3]={A(1,2),A(3,4),A(5,6)};
```

二、析构函数：死亡

- 没有参数
- 唯一
- 没有返回值

举例：

```
class MyClass {
    int num;
    int* ID_list;
public:
    MyClass() { // 构造函数
        // 构造函数体
    }
    ~MyClass() { // 析构函数
        // 析构函数体
        if(ID_list) delete[] ID_list; // 释放内存
    }
};
```

析构函数会在对象生命周期结束时自动调用。比如说程序结束的时候：

```
//我们此前 Test 里面创建了一个 Member m

Test t;
int main(){return 0;}
// 输出： ~Test() ~Member()
```

晚构造，先析构！！

析构函数也会自动隐式定义。但是不会删除指针成员，因此可能导致内存泄漏。

三、局部对象的构造和析构

其实没啥好说的，就是看括号里面的东西。

四、全局对象的构造和析构

- 在 main() 之前构造
- 在同一编译单元中，按照定义顺序构造
- 在不同编译单元中顺序不确定
- 在 main() 执行完 return 语句后析构

尽量少用全局对象！ 全局对象之间最好不要有依赖关系，否则析构顺序会出问题。